Cryptography and Network Security (BT14503) :

Advanced Encryption Standards

Dr. Shivangi Shukla

Assistant Professor

Computer Science and Engineering

IIIT Pune



AES

- Published by NIST (National Institute of Standards and Technology) in December 2001
  - First AES candidate conference
    - 15 out of 21 algorithms selected
  - Second AES candidate conference
    - 5 out of 15 selected as finalists
      - MARS, RC6, Rijndael, Serpent and Twofish
  - Third AES candidate conference
    - NIST announced Rijndael as the AES

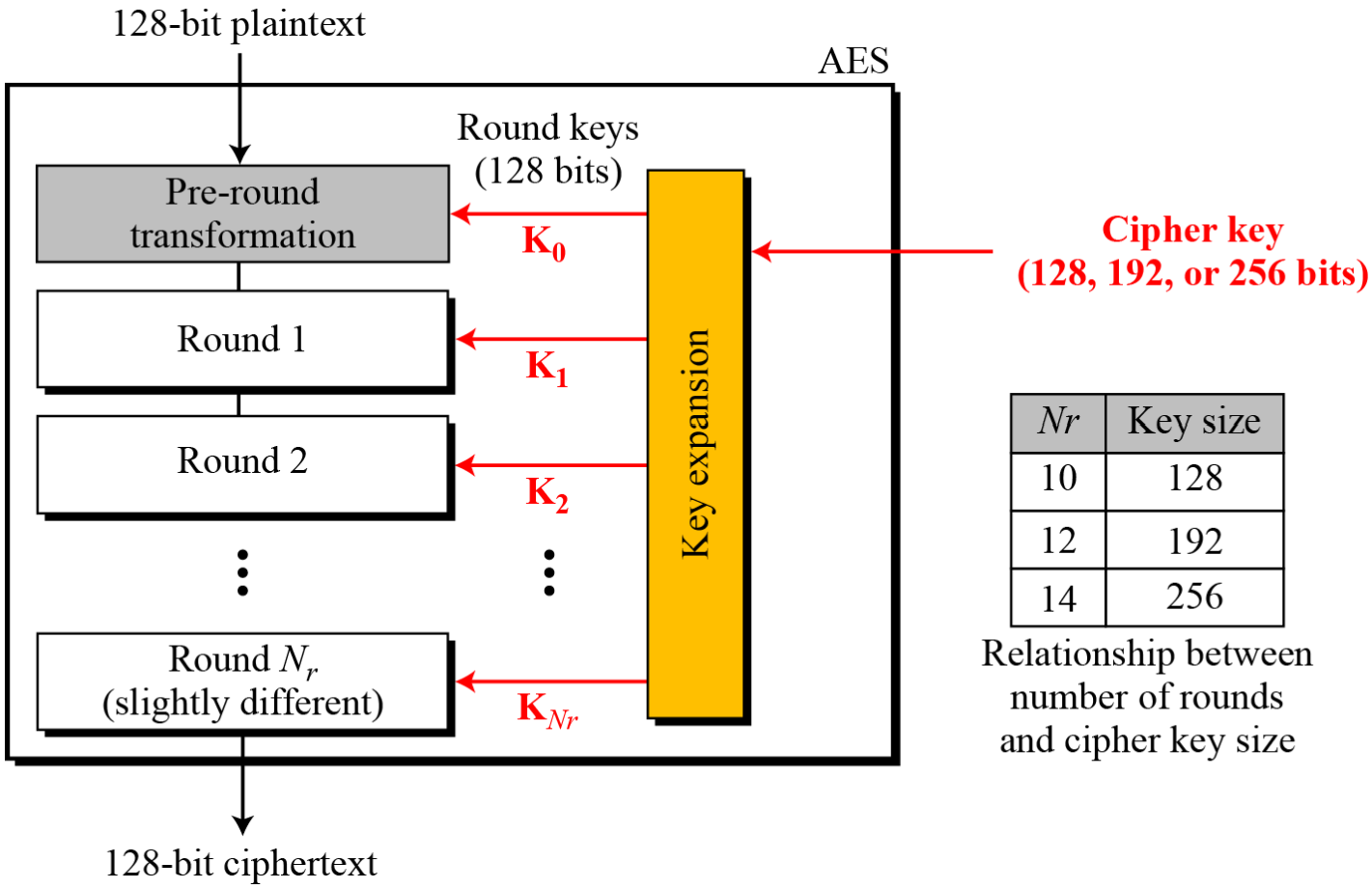
AES

- The criteria defined by NIST for selecting AES fall into three areas:
  - Security:
    - NIST explicitly demanded a 128-bit key, focused on resistance to cryptanalysis attacks other than brute-force attack.
  - Cost:
    - computational efficiency and storage requirement for different implementations such as hardware, software, or smart cards.
  - Implementation:
    - algorithm must have flexibility (be implementable on any platform) and simplicity.

AES

- AES is a non-Feistel cipher that encrypts and decrypts a data block of 128 bits.
  - It uses 10, 12, or 14 rounds.
- The key size
  - can be 128, 192, or 256 bits, depends on the number of rounds.
    - But the round keys are always 128 bits
- encryption algorithm
  - called cipher
- decryption algorithm
  - called inverse cipher

AES - General Design



Nr	Key size
10	128
12	192
14	256

Relationship between number of rounds and cipher key size

AES - General Design

- N_r: denotes number of rounds
- three different AES versions
  - referred as AES-128, AES-192, and AES-256
  - each version uses a different cipher key size (128, 192, or 256
    - However, round keys are always 128 bits
- The number of round keys generated by the key-expansion algorithm is always one more than the number of rounds
  - Number of round keys = $N_r + 1$
  - Round keys are referred as $K_0, K_1\ K_2 \ldots K_{N_r}$

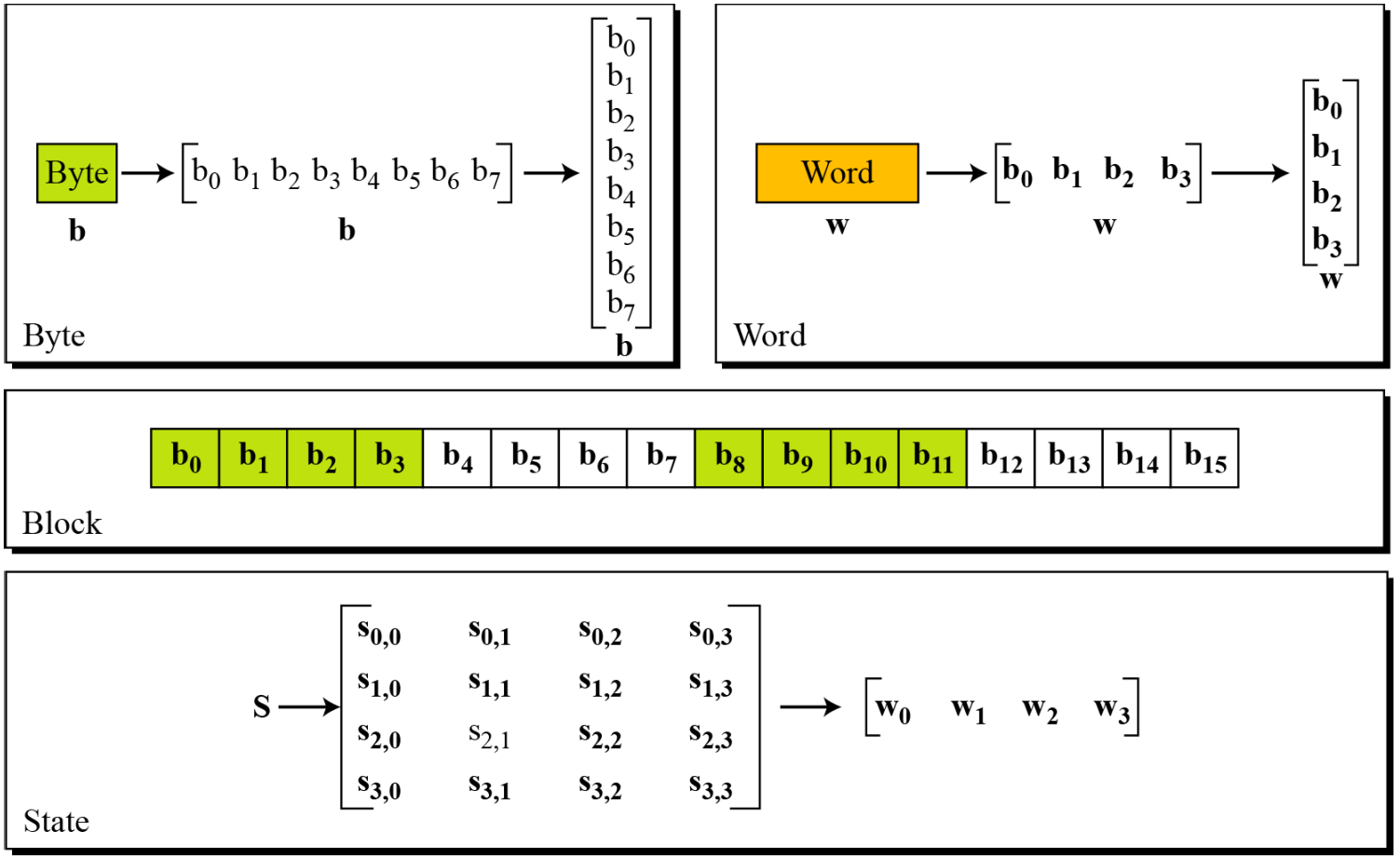
AES - Data Units

- AES uses five units of measurement to refer to data: bits, bytes, words, blocks, and state.
- Bit: binary digit with a value of 0 or 1.
  - lowercase letter b to refer to a bit.
- Byte: a group of eight bits that can be treated as a single entity,
  - a row matrix (1 × 8) of eight bits,
  - or a column matrix (8 × 1) of eight bits.
  - lowercase bold letter **b** to refer to a byte
- Word: a group of 32 bits that can be treated as a single entity,
  - a row matrix of four bytes,
  - or a column matrix of four bytes
  - lowercase bold letter **w** to show a word.

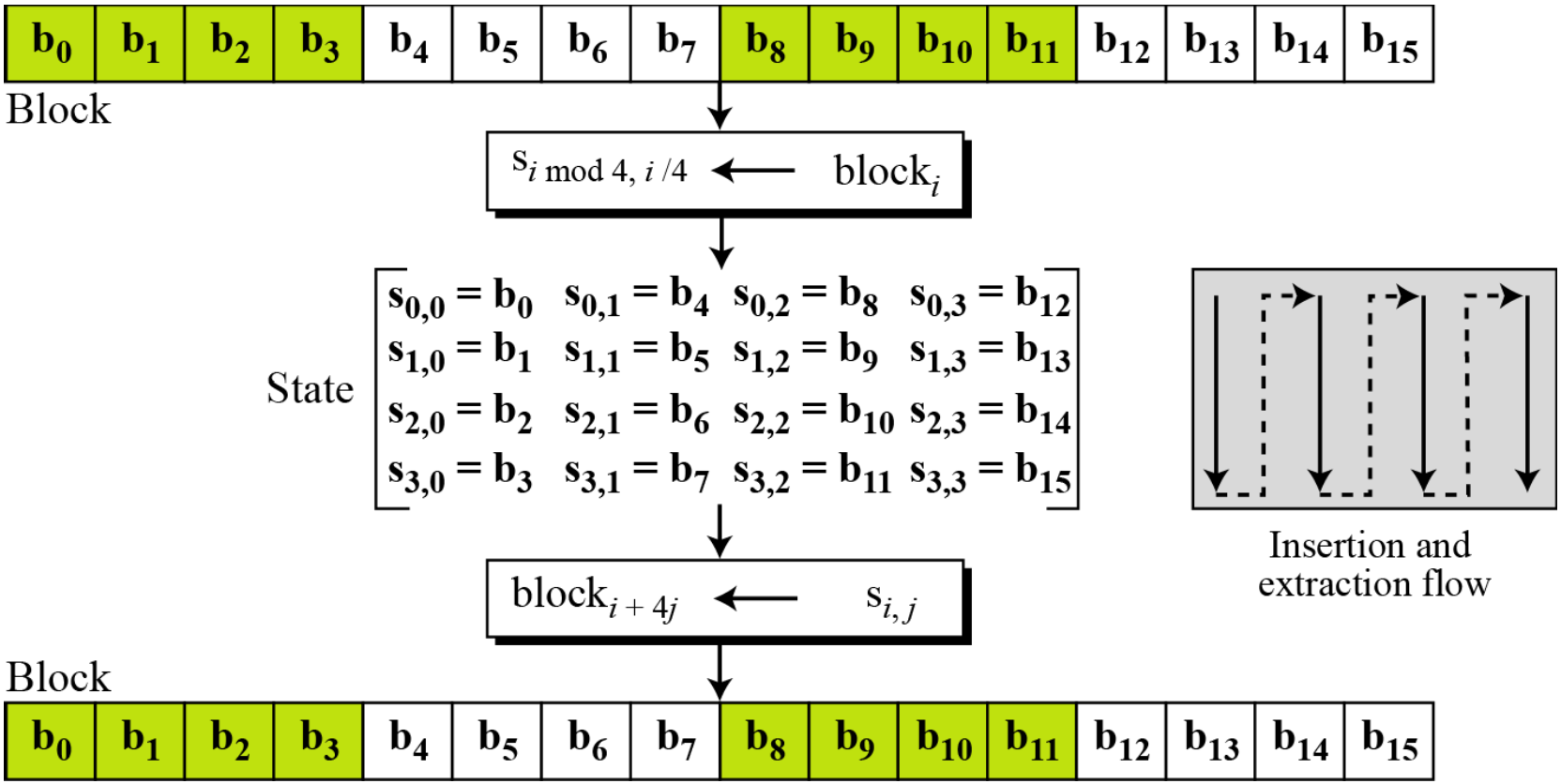
AES - Data Units

- Block
  - AES encrypts and decrypts data blocks.
  - A block in AES is a group of 128 bits.
    - However, a block can be represented as a row matrix of 16 bytes.
- State
  - each round is made of several stages
    - uppercase bold letter to refer to a state
  - Usually represented as **S**, occasionally represented as **T** to refer to a temporary state
  - States are made of 16 bytes, but normally are treated as **matrices** of **4 × 4 bytes**
    - each element of a state is referred to as $s_{r,c}$ where $r(0 \text{ to } 3)$ defines row and the c$(0 \text{ to } 3)$ defines column
  - Data block is transformed from one stage to another
    - data block is referred as a state

AES - Data Units



AES - Data Units



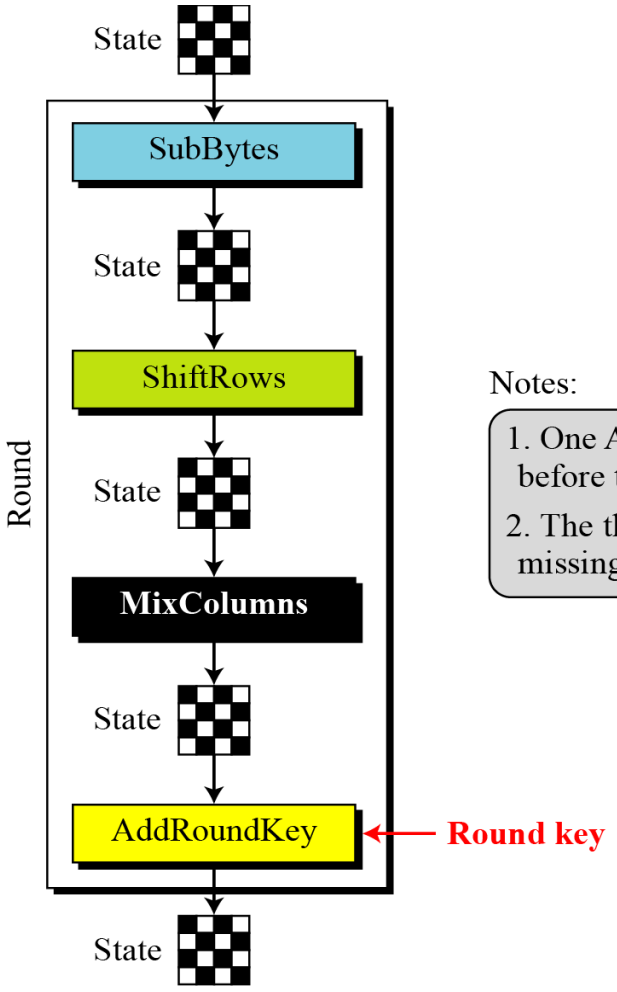
AES - Data Units

- Changing plaintext to states
  - replace each character with an integer between 00 and 25.
    - represent each byte as an integer with two hexadecimal digits

Text	A	E	S	U	S	E	S	A	M	A	T	R	I	X	Z	Z
Hexadecimal	00	04	12	14	12	04	12	00	0C	00	13	11	08	23	19	19

$$\begin{bmatrix} 00 & 12 & 0C & 08 \\ 04 & 04 & 00 & 23 \\ 12 & 12 & 13 & 19 \\ 14 & 00 & 11 & 19 \end{bmatrix}$$ State

Structure of each round



Notes:

1. One AddRoundKey is applied before the first round.
2. The third transformation is missing in the last round.

MixColumns is missing in the last round

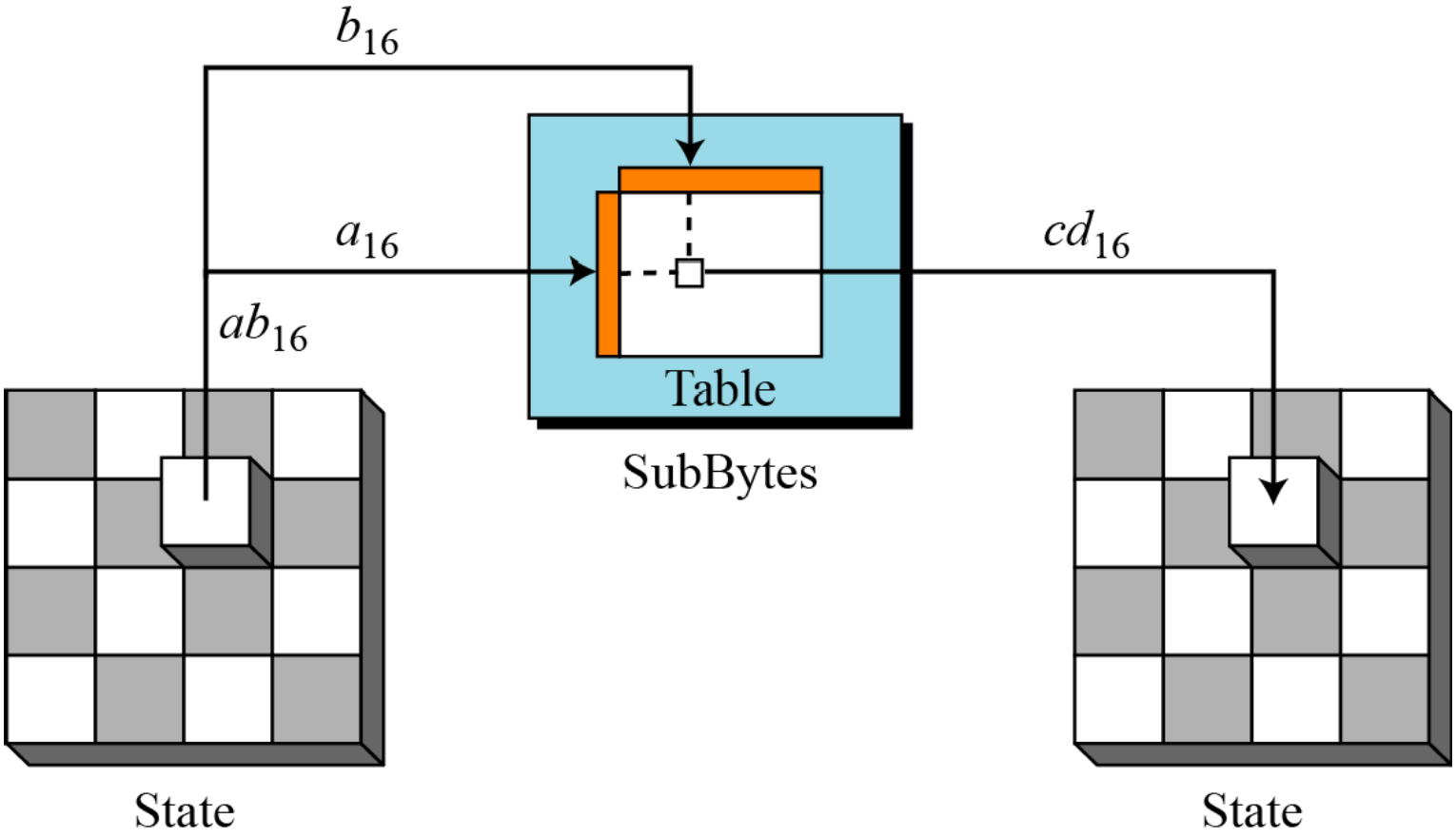
AES Transformations

- Four types
  - Substitution (SubBytes)
  - Permutation (ShiftRows)
  - Mixing (MixColumns)
  - Key-adding (AddRoundKey)

Substitution

- AES, like DES, uses substitution.
- SubBytes
  - The first transformation, SubBytes, is used in encryption .
    - InvSubBytes, inverse of SubBytes, is used in decryption
  - Provides confusion
  - To substitute a byte, we interpret the byte as two hexadecimal digits.
    - Left and right digit defines row and column, respectively in the substitution table.
  - The SubBytes operation involves 16 independent byte-to-byte transformations.

Substitution…



Substitution…

Table 1: SubBytes Transformation Table

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	CB	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

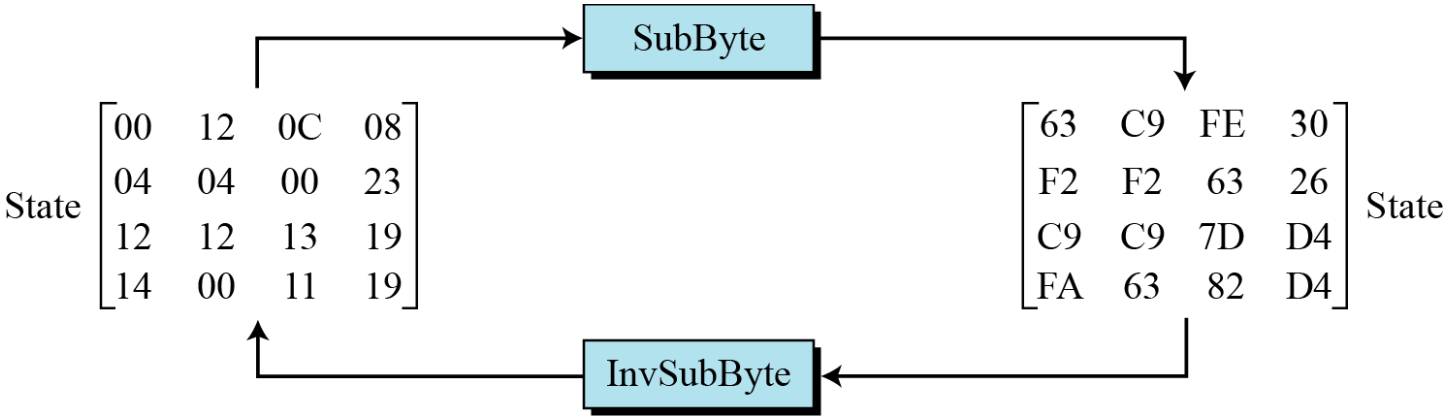
Substitution

Table 1: InvSubBytes Transformation Table

	0	*1*	*2*	*3*	*4*	*5*	*6*	*7*	*8*	*9*	*A*	*B*	*C*	*D*	*E*	*F*
0	52	09	6A	D5	30	36	A5	38	BF	40	A3	9E	81	F3	D7	FB
1	7C	E3	39	82	9B	2F	FF	87	34	8E	43	44	C4	DE	E9	CB
2	54	7B	94	32	A6	C2	23	3D	EE	4C	95	0B	42	FA	C3	4E
3	08	2E	A1	66	28	D9	24	B2	76	5B	A2	49	6D	8B	D1	25
4	72	F8	F6	64	86	68	98	16	D4	A4	5C	CC	5D	65	B6	92
5	6C	70	48	50	FD	ED	B9	DA	5E	15	46	57	A7	8D	9D	84
6	90	D8	AB	00	8C	BC	D3	0A	F7	E4	58	05	B8	B3	45	06
7	D0	2C	1E	8F	CA	3F	0F	02	C1	AF	BD	03	01	13	8A	6B
8	3A	91	11	41	4F	67	DC	EA	97	F2	CF	CE	F0	B4	E6	73
9	96	AC	74	22	E7	AD	35	85	E2	F9	37	E8	1C	75	DF	6E
A	47	F1	1A	71	1D	29	C5	89	6F	B7	62	0E	AA	18	BE	1B
B	FC	56	3E	4B	C6	D2	79	20	9A	DB	C0	FE	78	CD	5A	F4
C	1F	DD	A8	33	88	07	C7	31	B1	12	10	59	27	80	EC	5F
D	60	51	7F	A9	19	B5	4A	0D	2D	E5	7A	9F	93	C9	9C	EF
E	A0	E0	3B	4D	AE	2A	F5	B0	C8	EB	BB	3C	83	53	99	61
F	17	2B	04	7E	BA	77	D6	26	E1	69	14	63	55	21	0C	7D

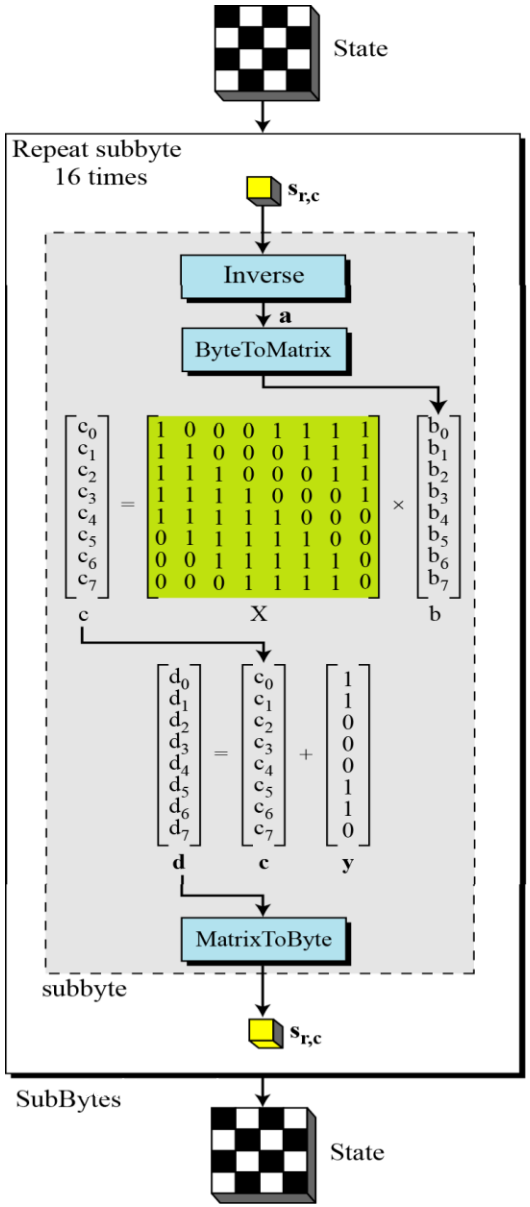
Substitution…

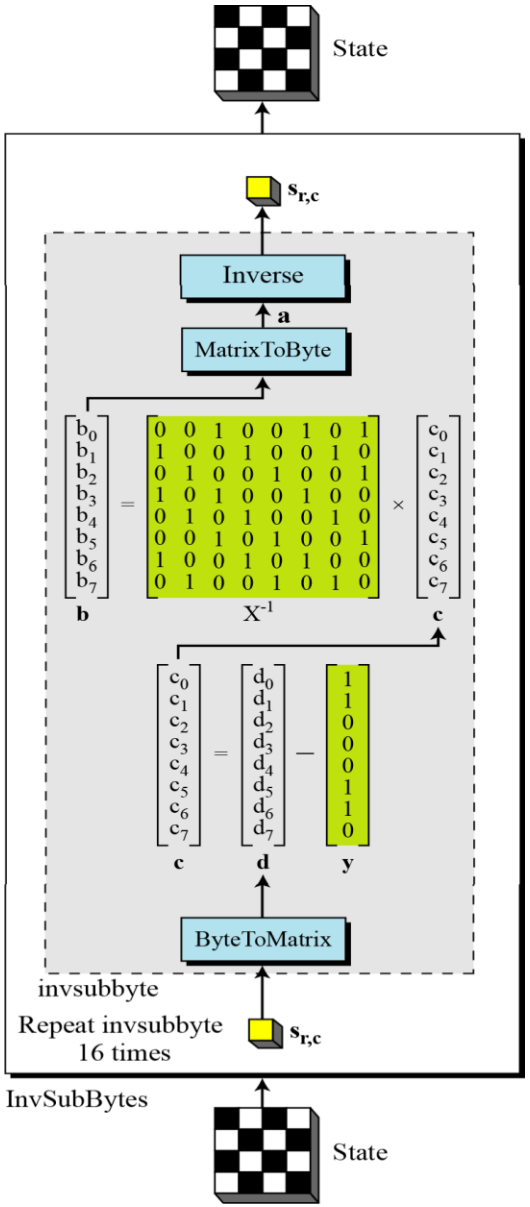
- Example



SubBytes and InvSubBytes processes

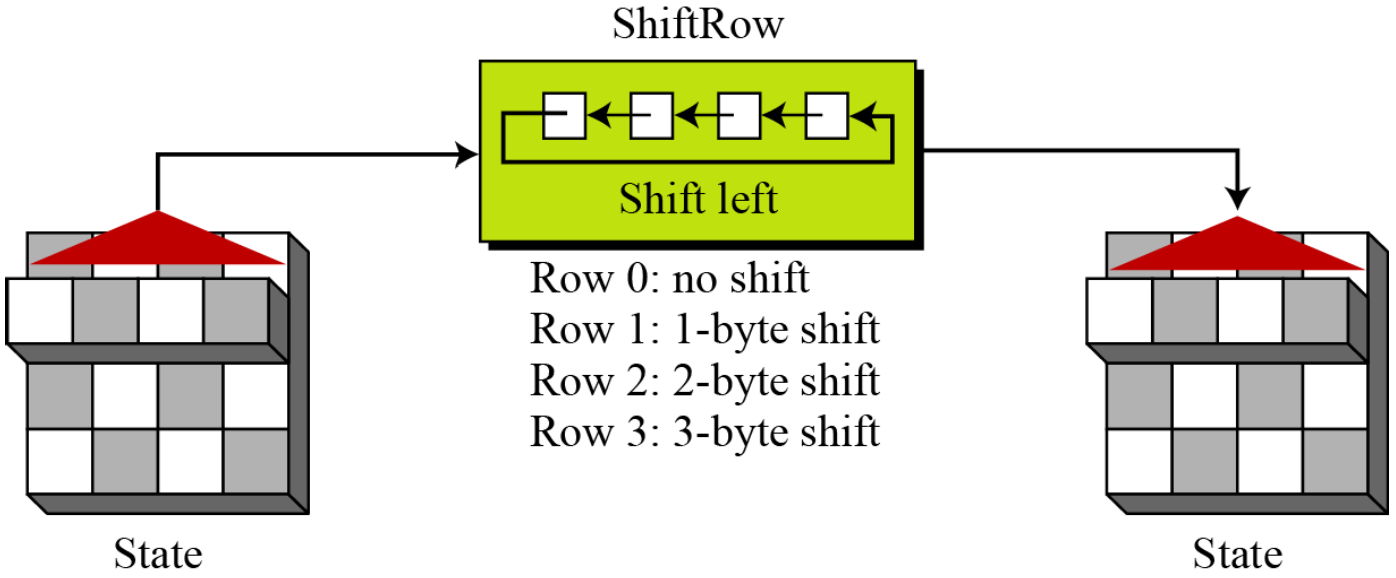
- AES also defines transformation algebraically using $\mathrm{GF}(2^8)$ field with irreducible polynomial $(x^8 + x^4 + x^3 + x + 1)$





Permutation

- ShiftRows
  - In the encryption, the transformation is called ShiftRows.
  - Shifting is done in left direction

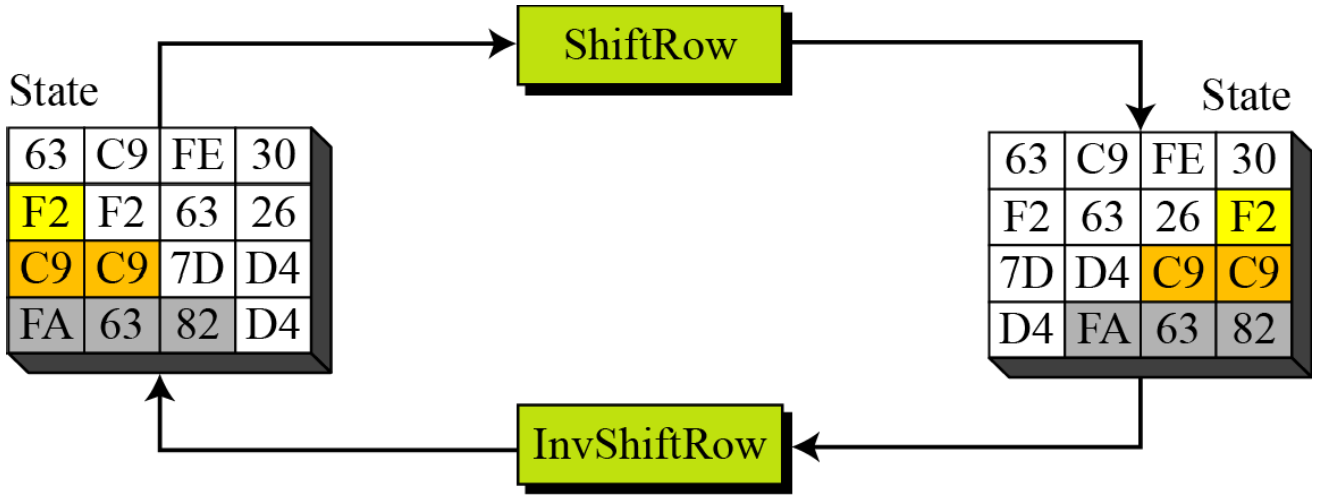


Permutation…

- InvShiftRows
  - In the decryption, the transformation is called InvShiftRows and the shifting is to the right.
  - Shifting is done in right direction
  - The number of shifts is same as row number (0,1,2, and 3) of state matrix
  - The ShiftRows and InvShiftRows transformation are inverses of each other

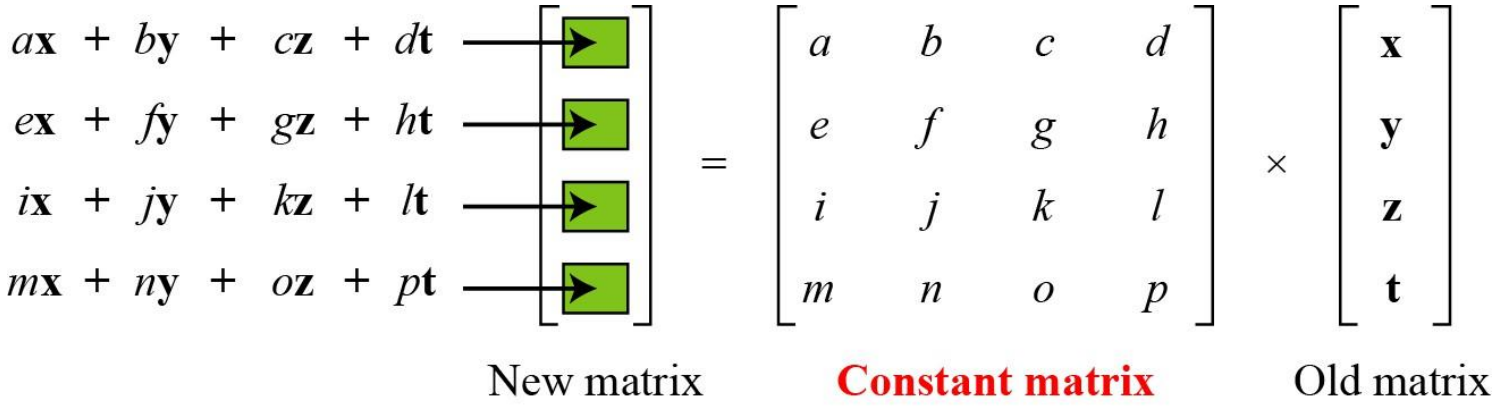
Permutation…

- Example



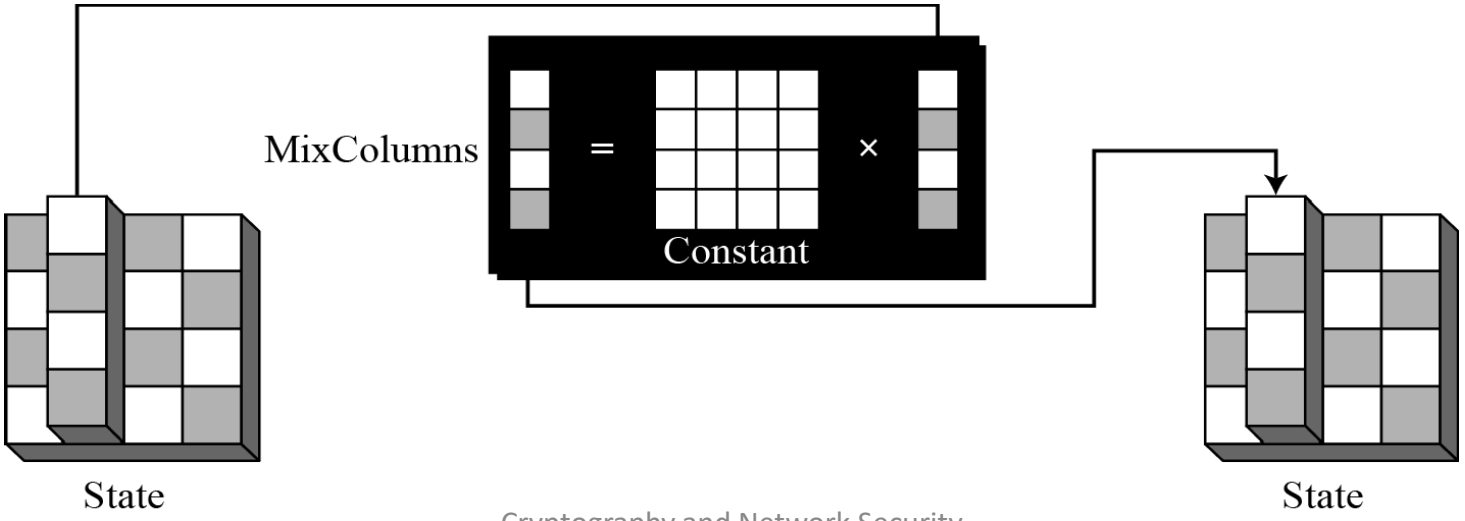
Mixing

- An interbyte transformation that changes the bits inside a byte, based on the bits inside the neighboring bytes.
  - We need to mix bytes to provide diffusion at the bit level.



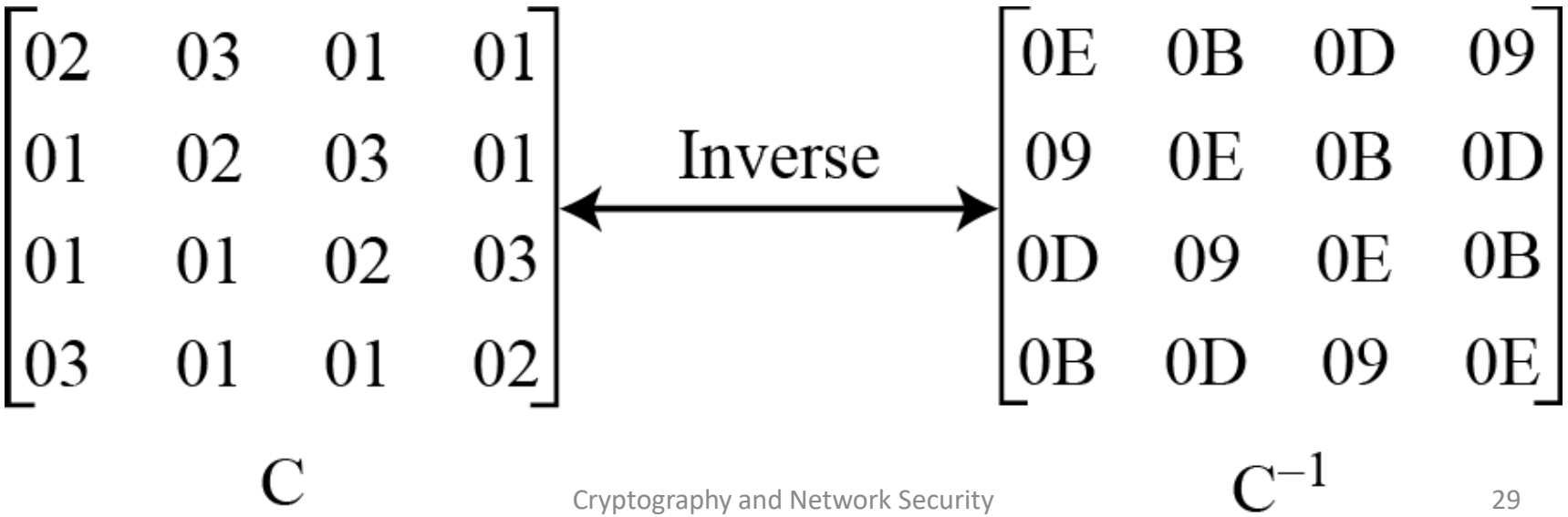
Mixing

- MixColumns
  - operates at the column level
  - transforms each column of the state to a new column.
  - Multiplication is done in $GF(2^8)$ with modulus (10001101) or $(x^8 + x^4 + x^3 + x + 1)$



Mixing

- InvMixColumns
  - same as the MixColumns transformation
    - but the inverse
  - For example,



Key Adding

- AddRoundKey
  - AddRoundKey proceeds one column at a time.
  - AddRoundKey adds a round key word with each state column matrix
  - the operation in AddRoundKey is matrix addition.

Key Adding..

- Note that
  - s_c and $w_{round+4c}$ are 4×1 column matrices.
  - $\oplus$ means XORing two column matrices, each of 4 bytes

Algo 1: Pseudocode for AddRoundKey transformation

```
AddRoundKey (S)
{
    for (c = 0 to 3)
        s_c ← s_c ⊕ w_(4 round + c)
}
```

Key Adding..

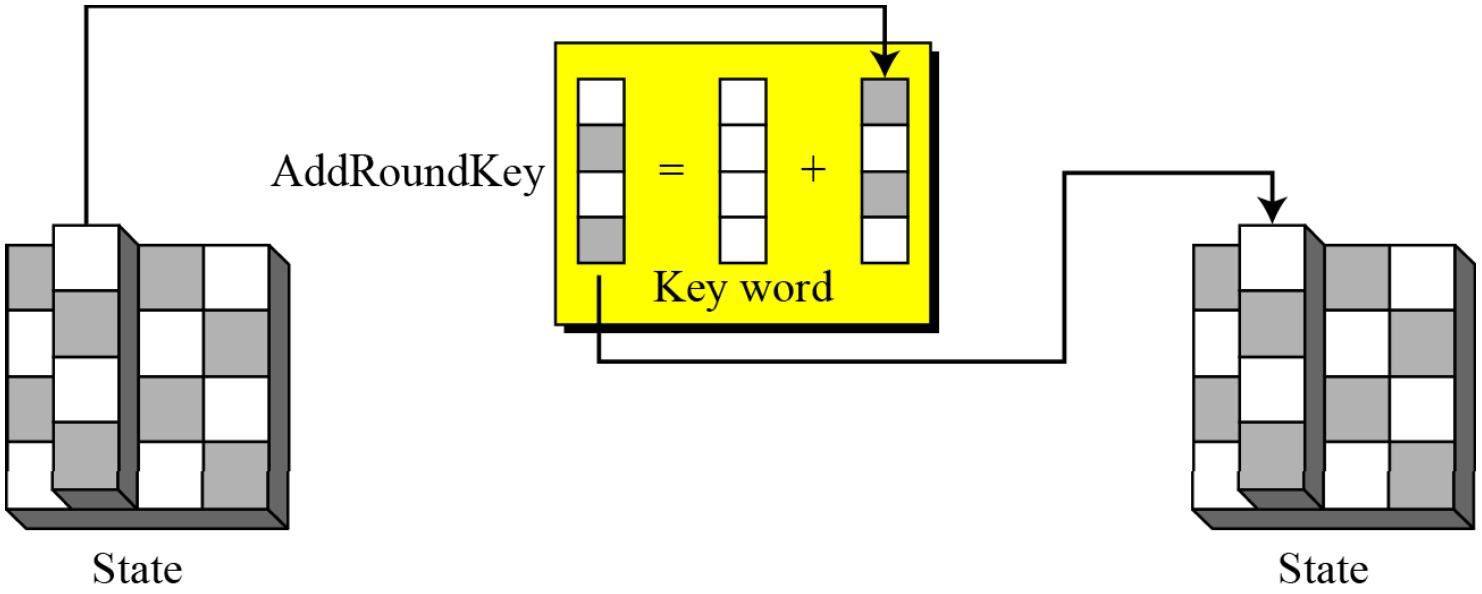


Fig : AddRoundKey transformation

Key Expansion

- To create round keys for each round, AES uses a key-expansion process.
- If the number of rounds is N_r , the key- expansion routine creates $N_r + 1$, 128-bit round keys from one single 128-bit cipher key.
- The first-round key is used for pre-round transformation
  - The remaining round keys are used for last transformation (AddRoundKey) at end of each round

Key Expansion..

- The key-expansion routine creates round keys word by word,
  - where a word is an array of four bytes.
- The routine creates $4 \times (N_r + 1)$ words:

  $w_0, w_1, w_2, w_3, \ldots\ldots w_{4(N_r+1)-1}$
- AES-128
  - has 10 rounds
  - 44 words
- AES-192
  - has 12 rounds
  - 52 words
- AES-256
  - has 14 rounds
  - 60 words

Key Expansion..

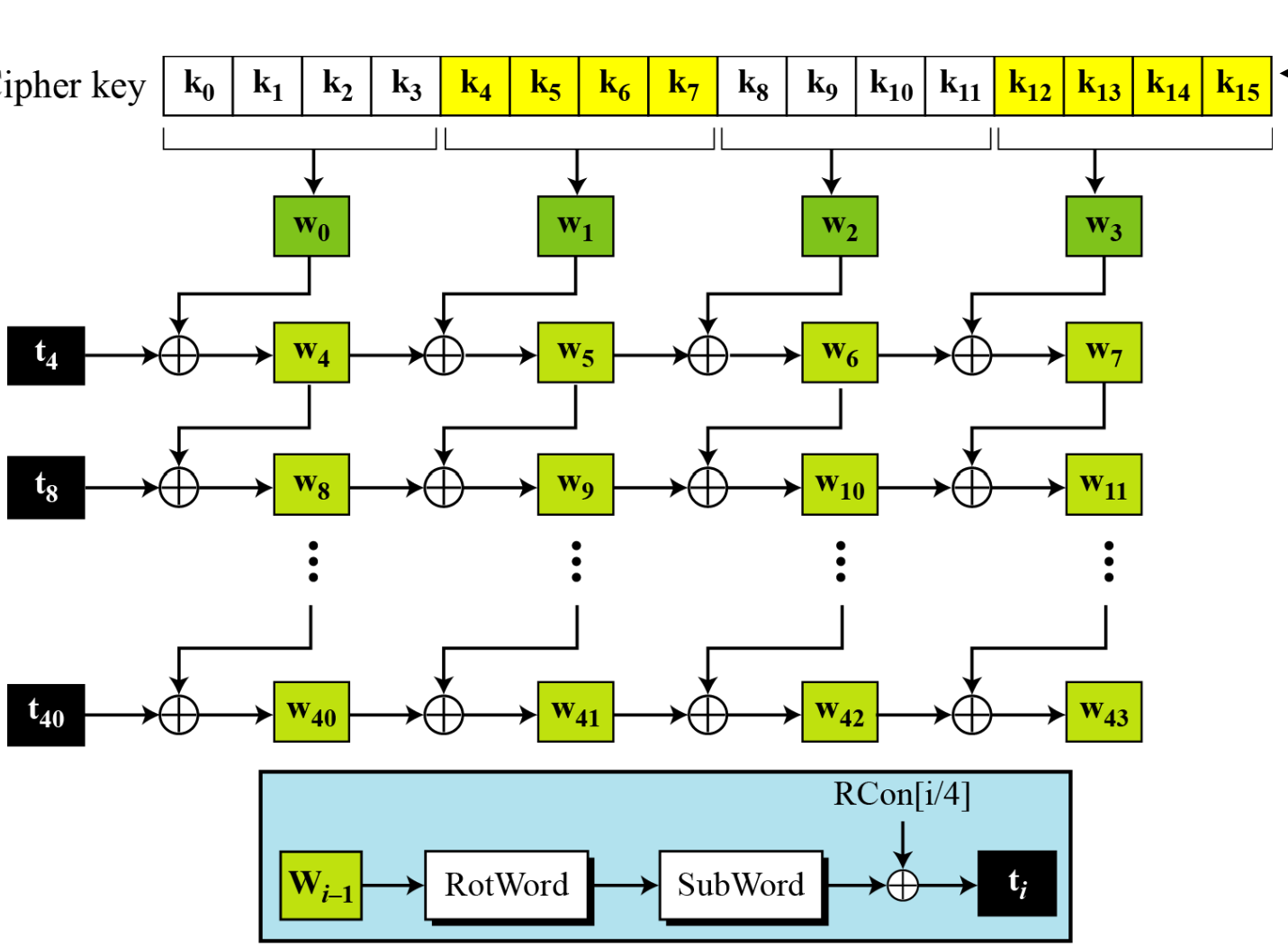
- The key-expansion routine creates round keys word by word,
  - where a word is an array of four bytes.
- The routine creates $4 \times (N_r + 1)$ words:

$w_0, w_1, w_2, w_3, \ldots\ldots w_{4(N_r+1)-1}$

Table : Words for each round

Round	*Words*			
Pre-round	$\mathbf{w}_0$	$\mathbf{w}_1$	$\mathbf{w}_2$	$\mathbf{w}_3$
1	$\mathbf{w}_4$	$\mathbf{w}_5$	$\mathbf{w}_6$	$\mathbf{w}_7$
2	$\mathbf{w}_8$	$\mathbf{w}_9$	$\mathbf{w}_{10}$	$\mathbf{w}_{11}$
. . .	. . .			
N_r	$\mathbf{w}_{4N_r}$	$\mathbf{w}_{4N_r+1}$	$\mathbf{w}_{4N_r+2}$	$\mathbf{w}_{4N_r+3}$

Key Expansion in AES-128..



Making of t_i (temporary) words $i = 4\ N_r$

Key Expansion in AES-128..

- The first four words (w_0, w_1, w_2, w_3) are made from the cipher key.
  - The cipher key is thought of as an array of 16 bytes (k_0 to k_{15}) where (k_0 to k_3) becomes w_0 ;
  - next four bytes (k_4 to k_7) becomes w_1 and so on.
- The rest of w_i for $i = 4\ to\ 43$:
  - If $(i\ mod\ 4) \neq 0$ then $w_i = w_{i-1} \bigoplus w_{i-4}$
  - If $(i\ mod\ 4) = 0$ then $w_i = t \bigoplus w_{i-4}$, where t is a temporary word, result of applying SubWord and RotWord on w_i and XORing the result with round constants RCon

Key Expansion in AES-128..

- $t = \text{SubWord}(\text{RotWord}(w_{i-1})) \oplus \text{RCon}_{i/4}$
- RotWord
  - rotate word
  - takes a word as an array of four bytes
  - and shifts each byte to left with wrapping
- SubWord
  - substitute word
  - applied to only four bytes.
  - takes each byte in the word and substitutes another byte for it

Key Expansion in AES-128..

- RCon is 4-byte value in which rightmost three bytes are always 0

Table 7.4 *RCon constants*

Round	*Constant (RCon)*	*Round*	*Constant (RCon)*
1	$(\underline{\mathbf{01}}\ 00\ 00\ 00)_{16}$	6	$(\underline{\mathbf{20}}\ 00\ 00\ 00)_{16}$
2	$(\underline{\mathbf{02}}\ 00\ 00\ 00)_{16}$	7	$(\underline{\mathbf{40}}\ 00\ 00\ 00)_{16}$
3	$(\underline{\mathbf{04}}\ 00\ 00\ 00)_{16}$	8	$(\underline{\mathbf{80}}\ 00\ 00\ 00)_{16}$
4	$(\underline{\mathbf{08}}\ 00\ 00\ 00)_{16}$	9	$(\underline{\mathbf{1B}}\ 00\ 00\ 00)_{16}$
5	$(\underline{\mathbf{10}}\ 00\ 00\ 00)_{16}$	10	$(\underline{\mathbf{36}}\ 00\ 00\ 00)_{16}$

Key Expansion in AES-128..

Algo 2: Pseudocode for key expansion in AES-128

```
KeyExpansion ([key0 to key15], [w0 to w43])
{
    for (i = 0 to 3)
        wi ← key4i + key4i+1 + key4i+2 + key4i+3

    for (i = 4 to 43)
    {
        if (i mod 4 ≠ 0)    wi ← wi–1 + wi–4
        else
        {
            t ← SubWord (RotWord (wi–1)) ⊕ RCon i/4        // t is a temporary word
            wi ← t + wi–4
        }
    }
}
```

Key Expansion in AES-128..

- What about weak keys for AES???

Pre-round:	00000000	00000000	00000000	00000000
Round 01:	62636363	62636363	62636363	62636363
Round 02:	9B9898C9	F9FBFBAA	9B9898C9	F9FBFBAA
Round 03:	90973450	696CCFFA	F2F45733	0B0FAC99
...	...	...	...	...
Round 10:	B4EF5BCB	3E92E211	23E951CF	6F8F188E

Key Expansion in AES-192

- In AES-192,
  - the words are generated in groups of six
- The cipher key creates the first six words w_0 to w_5
  - If $(i \ mod \ 6) \neq 0$ then $w_i = w_{i-1} \oplus w_{i-6}$
  - If $(i \ mod \ 6) = 0$ then $w_i = t \oplus w_{i-6}$
    - where $t =$ SubWord(RotWord(w_{i-1})) $\oplus$ $\text{RCon}_{i/6}$

Key Expansion in AES-256

- In AES-256,
  - the words are generated in groups of eight
- The cipher key creates the first eight words w_0 to w_7
  - If $(i \ mod \ 8) \neq 0$ then $w_i = w_{i-1} \oplus w_{i-8}$
  - If $(i \ mod \ 4) = 0$ and $(i \ mod \ 8) \ \neq 0$ then
    - $w_i = SubWord(w_{i-1}) \oplus w_{i-8}$
  - Otherwise $w_i = t \oplus w_{i-8}$
    - where $t =$ SubWord(RotWord(w_{i-1})) $\oplus$ $\text{RCon}_{i/8}$

Key Expansion

- Each round key in AES depends on the previous round key.
  - The dependency, however, is nonlinear because of SubWord transformation.
- The addition of the round constants guarantees
  - that each round key will be different from the previous one.
- An illustration of cipher key in AES-128

Cipher Key 1: 12 45 A2 A1 23 31 A4 A3 B2 CC A**A** 34 C2 BB 77 23
Cipher Key 2: 12 45 A2 A1 23 31 A4 A3 B2 CC A**B** 34 C2 BB 77 23

Key Expansion in AES-128..

Table 7.6 *Comparing two sets of round keys*

R.	Round keys for set 1	Round keys for set 2	B. D.
—	1245A2A1 2331A4A3 B2CCAA34 C2BB7723	1245A2A1 2331A4A3 B2CCAB34 C2BB7723	01
1	F9B08484 DA812027 684D8A13 AAF6FD30	F9B08484 DA812027 684D8B13 AAF6FC30	02
2	B9E48028 6365A00F 0B282A1C A1DED72C	B9008028 6381A00F 0BCC2B1C A13AD72C	17
3	A0EAF11A C38F5115 C8A77B09 6979AC25	3D0EF11A 5E8F5115 55437A09 F479AD25	30
4	1E7BCEE3 DDF49FF6 1553E4FF 7C2A48DA	839BCEA5 DD149FB0 8857E5B9 7C2E489C	31
5	*EB2999F3 36DD0605 238EE2FA 5FA4AA20*	*A2C910B5 7FDD8F05 F78A6ABC 8BA42220*	34
6	82852E3C B4582839 97D6CAC3 C87260E3	CB5AA788 B487288D 430D4231 C8A96011	56
7	82553FD4 360D17ED A1DBDD2E 69A9BDCD	588A2560 EC0D0DED AF004F1C 67A92FCD	50
8	D12F822D E72295C0 46F948EE 2F50F523	0B9F98E5 E7929508 4892DAD4 2F3BF519	44
9	99C9A438 7EEB31F8 38127916 17428C35	F2794CF0 15EBD9F8 5D79032C 7242F635	51
10	03AD32C8 FD460330 C5547A26 D216F613	E03BDAB0 FDD00348 A0A90064 D2EBF651	52